

Enlight Lab

Manager Demo Cheat Sheet

Browser UI demo for your manager.

Paste commands to Cursor - AI runs them behind the scenes.

Hide PowerShell and kubectl from the screen.

Project: D:\enlight-lab-platform

Repo: github.com/kirtiprasad2003/enlight-lab-platform

BEFORE THE MEETING (you only - hide terminal)

Run 15-30 minutes before the call. Minimize PowerShell.

```
cd D:\enlight-lab-platform
.\scripts\go-live.bat
```

Quick check: <http://localhost:30800/health> must show {"status":"ok"}

If connection refused -> run go-live.bat again.

BROWSER TABS (share screen on these)

#	URL	Notes
1	http://localhost:30800	Enlight Lab dashboard
2	http://localhost:30800/health	JSON health check
3	http://localhost:8082	ArgoCD -> fastapi-staging
4	http://localhost:3000	Grafana admin / enlight-admin
5	github.com/.../actions	GitHub Actions tab
6	platform.robusta.dev	Robusta (optional)

DO NOT share: PowerShell, kubectl, or Cursor terminal.

OPENING LINE (30 seconds)

"You saw two earlier PoCs - delivery and policy separately. Enlight Lab unifies them on one platform: pipeline, policy gates, GitOps, and monitoring. Today it runs at zero cloud cost locally; production uses the same design on EKS."

PART 1 - Live platform (~2 min) - BROWSER ONLY

Show:

- `http://localhost:30800` - dashboard UI
- `http://localhost:8082` - ArgoCD fastapi-staging green/Synced
- `http://localhost:30800/health` - `{"status":"ok"}`

Say:

"The app is live on Kubernetes. ArgoCD keeps it in sync with Git."

Cursor command: None

PART 2 - Demo 2: BLOCK bad deploy (~2 min)

CI policy rejects bad manifest before it reaches the cluster.

Paste to Cursor:

```
Dispatch chat-to-deploy non-compliant on kirtiprasad2003/enlight-lab-platform main
```

Show in browser:

- GitHub Actions -> Chat to Deploy -> RED failed policy-check
- Violation: :latest tag, bad registry, missing limits

Say:

"Bad configuration is stopped in CI. It never touches production."

PART 3 - Demo 2: PASS good deploy (~2 min)

Paste to Cursor:

```
Dispatch chat-to-deploy compliant on kirtiprasad2003/enlight-lab-platform main
```

Show:

- GitHub Actions -> GREEN run
- ArgoCD still healthy
- `/health` still ok

Say:

"Compliant config passes policy and the app stays healthy."

PART 4 - Demo 1: Incident response (~4 min)

Step 4a - Break the app (on purpose)

Paste to Cursor:

```
Run demo 1: inject failure on fastapi in enlight-staging
```

Show:

- ArgoCD -> pod RED / degraded
- Robusta alert if visible

Say:

"Something broke on purpose. In production this could be a bad image push."

Step 4b - AI explains (do NOT fix yet)

Paste to Cursor:

```
Use k8sgpt to explain the fastapi failure in enlight-staging
```

Alternative: Explain why the fastapi pod is failing in enlight-staging

Say:

"AI diagnoses the incident. Explanation is separate from the fix."

Step 4c - Heal / rollback

Paste to Cursor:

```
Heal demo 1 - rollback fastapi to last good image in enlight-staging
```

Show:

- ArgoCD recovers to green
- /health ok again

Say:

"GitOps rolls back to last good version. AI explains; ArgoCD fixes."

PART 5 - Observability (~1 min) - BROWSER ONLY

Show: <http://localhost:3000> (Grafana)

"Prometheus and Grafana give SLOs and metrics for rollback decisions."

Cursor command: None

PART 6 - Demo 5: PR compliance (optional ~2 min)

Paste to Cursor:

```
Show me the latest PR compliance workflow run on enlight-lab-platform
```

Or: Create a demo PR with non-compliant samples and show failed checks

Show:

- GitHub PR -> Checks -> blocked vs passed

Say:

"Every PR is scanned for secrets and insecure infra."

If no PR ready: say same bot runs on every pull request.

PART 7 - Demo 3: IDP scaffold (~1 min)

Paste to Cursor:

```
Run demo 3 scaffold demo-api and show me what was created
```

Show:

- Folder workload/scaffolded/demo-api in repo

Say:

"In production this is Backstage - one click scaffolds a compliant bundle."

PART 8 - Demo 4: Drift (mention only ~30 sec)

No browser UI. Say briefly:

"Demo 4 detects AWS changes outside Terraform, estimates cost, and reconciles back to Git. Runs locally with simulated AWS."

CLOSING (~30 sec)

"Five demos on one platform: CI policy, GitOps, monitoring, AI incident response, and PR compliance. Zero cloud cost now - same architecture on EKS in production."

FULL COMMAND LIST (copy-paste in order)

```
# Part 1 - browser only

# Part 2 - BLOCK
Dispatch chat-to-deploy non-compliant on kirtiprasad2003/enlight-lab-platform main

# Part 3 - PASS
Dispatch chat-to-deploy compliant on kirtiprasad2003/enlight-lab-platform main

# Part 4 - Incident
Run demo 1: inject failure on fastapi in enlight-staging
Use k8sgpt to explain the fastapi failure in enlight-staging
Heal demo 1 - rollback fastapi to last good image in enlight-staging

# Part 5 - Grafana (browser only)

# Part 6 - PR compliance (optional)
Show me the latest PR compliance workflow run on enlight-lab-platform

# Part 7 - IDP scaffold
Run demo 3 scaffold demo-api and show me what was created
```

IF SOMETHING BREAKS

Problem	Fix (private)
:30800 connect refused	Run scripts\go-live.bat
ArgoCD cli not found	Run scripts\go-live.bat
App broken	Run demo 1 - rollback fastapi
Dashboard broken	Run scripts\go-live.bat

Rebuild dashboard UI (before demo, private):

```
cd D:\enlight-lab-platform
docker build -t enlight-fastapi:demo-pass workload/fastapi
kind load docker-image enlight-fastapi:demo-pass --name enlight-lab
kubectl rollout restart deployment/fastapi -n enlight-staging
.\scripts\go-live.bat
```

IF ASKED: "Is it complete?"

"Foundation and all five demos are runnable locally. Demos 1 and 2 are production-depth; 3-5 use the same foundation. Remaining work is polish, not architecture."

LIKELY QUESTIONS

Why not AWS today?

Local kind = \$0. EKS Terraform ready for production.

What does AI do?

Explains incidents. ArgoCD performs rollback - separate steps.

What is Floci?

Simulated local AWS for Demo 4 drift - not for Kubernetes.